Guidelines for Practical Project

Engineering of Intelligent Models

1 Introduction

This document describes the structure, expectations, and assessment criteria for the semester project in the Engineering of Intelligent Models curricular unit. The practical work will be developed progressively in three stages, allowing students to explore the full lifecycle of machine learning systems. Students will transition from experimental code to a production-ready system by implementing data versioning, experiment tracking, configuration management, pipeline orchestration, and model serving. Key technologies and tools / frameworks include **DVC**, **MLflow**, **Hydra**, **Apache Airflow**, and **FastAPI / LitServe**.

The primary objective of this project is not to achieve state-of-the-art predictive accuracy, but rather to construct a robust, scalable, and fully reproducible MLOps pipeline. Project evaluation will highly value incremental development, structural organization, reproducibility, strict data lineage, and code quality.

2 Assessment Structure

2.1 Tutorial #1 – Project Foundations, DVC, and MLflow Integration

For the first submission, students must define their project topic and establish the foundational MLOps infrastructure. The submission must include a clearly structured repository, demonstrating the separation of data, source code, and configuration. Students are expected to provide a functional baseline script for data ingestion (can be local or via external APIs). Crucially, this stage requires clear evidence of **DVC** implementation and its integration with **MLflow**. The submission must demonstrate strict **data lineage**, where an MLflow experiment run explicitly logs the DVC content hash (MD5) of the exact dataset version used. A **short written document** describing the chosen dataset, the machine learning problem, and the early MLOps (expected) project pipeline diagram is also required (**máx. 1-2 pages!**).

2.2 Tutorial #2 – Configuration Management and Pipeline Orchestration

This second submission builds upon the reproducible foundation by introducing modularity and automation. Students are expected to integrate **Hydra** to eliminate hardcoded parameters, managing API configurations, file paths, and model hyperparameters via hierarchical YAML files.

Furthermore, students must orchestrate their isolated scripts into a Directed Acyclic Graph (DAG) using **Apache Airflow**. The pipeline must automatically execute the complete workflow: Data Ingestion $\rightarrow$ Data Preprocessing $\rightarrow$ Training $\rightarrow$ Evaluation. The submission must include evidence (e.g., screenshots or logs) of the Airflow scheduler successfully triggering and completing the pipeline, with all metadata correctly routed to the MLflow tracking service.

2.3 Final Delivery – Model Serving and End-to-End MLOps Pipeline

The final stage consists of wrapping the orchestrated pipeline into a fully deployable system, accompanied by a comprehensive written report. The final solution must include:

- The final automated Airflow pipeline that registers the best-performing model into the **MLflow Model Registry**;
- A robust **FastAPI** serving layer that dynamically loads the production-ready model from the registry and exposes it via RESTful endpoints;
- A **Minimal Working Frontend** (e.g., built with Streamlit, Gradio, etc.) that interacts with the FastAPI backend, allowing end-users to input features and receive real-time predictions;
- A **Demonstration Pipeline** showcasing the end-to-end workflow: from updating a configuration in Hydra, triggering an Airflow run, verifying the new model in MLflow, to querying the updated API.

Students are encouraged to explore containerization (Docker Compose) to launch their entire infrastructure seamlessly.

3 Project Topics and Exploration Guidelines

Unlike High Performance Graphics Computing curricular unit, there will be no predefined list of datasets or algorithms. Students are free to structure a project **from scratch** on a topic of their choice or **reuse datasets and models from previous curricular units** (e.g., Machine Learning, Deep Learning - you choose!).

The underlying algorithm can range from traditional models (Random Forests, Gradient Boosting) to deep learning architectures (CNNs, LSTMs, etc.). The academic focus must remain entirely on the *engineering* of the pipeline surrounding the model (or models, if you decide to apply different predictors to guarantee best decision possible). A simple logistic regression wrapped in a flawless, automated, and versioned MLOps pipeline will receive a significantly higher grade than a complex neural network with poor traceability and manual execution steps.

4 Requirements

- Each group must clearly articulate their machine learning problem and build a corresponding **MLOps-friendly folder structure**.
- The project must enforce **100% reproducibility**. The professor should be able to **clone the group Git repository**, pull the data via DVC (if simulated "remote" folder, ZIP your project folder and submit via Moodle), and run the pipeline to achieve the exact same results logged in MLflow.
- Execution parameters must be managed dynamically via **Hydra**, and other configs that you see fit for Hydra.
- Workflows must be orchestrated via **Apache Airflow**, transitioning away from manual script execution.
- The final model(s) must be served via **FastAPI / LitServer** and accessible through a functional frontend.
- Code should be clear, adequately documented, and managed via Git.

5 Group Composition

- The project will be carried out in groups of **3-4 students**.
- Final assessment and oral discussion will be performed **individually**, ensuring each member comprehensively understands the MLOps concepts, implementation decisions, and infrastructure architecture.

6 Submission

All tutorial submissions (which should be named `Tutorial1`, `Tutorial2`, and `FinalProject`) must be uploaded to Moodle in a compressed format (`.ZIP`, `.RAR`, or `.TAR.GZ`) named **EMI-XX_SubmssName** where *XX* represents the group number. If the repository or data cache exceeds Moodle upload size limits, students may submit via **OneDrive**, including a text file with access instructions. Only the **final upload before the deadline** will be considered for evaluation.

6.1 Requirements for each submission

- **Tutorial #1**: Written document outlining the project idea, alongside a compressed file (or repository link) containing the codebase, demonstrating DVC tracking and MLflow integration.
- **Tutorial #2**: Updated codebase containing Hydra configuration structures and Apache Airflow DAG definitions, proving pipeline orchestration.
- **Final Project**: The fully developed project, containing the FastAPI / LitServe application, frontend, complete orchestration, and a final written report documenting the MLOps architecture (**máx. 15 pages!**).

6.2 Deadlines

- Tutorial #1: **15/03/2026**
- Tutorial #2: **29/03/2026**
- Final Delivery: **12/04/2026**
- Final Individual Discussion: **21/04/2026**

7 Oral Discussion

- The final oral defence will take place on the scheduled dates announced in Moodle.
- Each discussion session lasts approximately **20–25 minutes**.
- Only **one laptop per group** is necessary to present the project. The whole infrastructure (via Docker Compose) must be pre-launched and ready to demonstrate.
- Grades will not be announced immediately; assessment will only be finalised after reviewing all submissions and evaluating individual technical proficiency during the discussion.

8 Evaluation Criteria

- **CA1**: Adherence to MLOps repository standards, documentation, and clean code practices.
- **CA2**: Effectiveness of DVC and MLflow integration to ensure strict data lineage and experiment tracking.
- **CA3**: Correct use of Hydra for dynamic parameters and other configuration variables and definitions.
- **CA4**: Robustness of the Airflow DAG implementation, pipeline automation and completion, and routing to MLflow.
- **CA5**: Quality of the FastAPI serving layer, integration with the MLflow Model Registry, and functionality of the frontend.
- **CA6**: Quality of the written reports, highlighting the whole end-to-end MLOps pipeline, their components and relevance of the written content to this project.
- **CA7**: Technical proficiency, theoretical understanding, and clarity of explanation during the oral discussion.

9 Final Notes

Projects should remain technically demanding on the engineering side. Students are highly encouraged to experiment beyond the minimum specifications.

10 Additional Information

- Maximum grade: 20 points.
- Grade Distribution:
  - Tutorial #1: 20%
  - Tutorial #2: 40%
  - Final Delivery: 40%